

# README.md

---

## Legacy Modernization Audit: Vulnerable-Node-Source

---

### Project Overview

---

This is a legacy E-Commerce application built using **Node.js (Express)**, **EJS templates**, and **PostgreSQL**. It provides basic functionality for user authentication, product browsing, searching, and purchasing. The codebase appears to be an experimental or educational project ("Vulnerable Node") likely designed to demonstrate security flaws, but we are treating it as a legacy system to be modernized.

### Architectural Verdict

---

#### Current State: Monolithic Layered Architecture (Degraded)

The application follows a standard Express.js pattern (Router → Controller → Model), but exhibits "Big Ball of Mud" characteristics:

- SQL Injection Vulnerabilities:** The data layer constructs SQL strings via concatenation ( `model/products.js` ), making it inherently insecure and difficult to maintain.
- Environment Hardcoding:** Configuration logic ( `config.js` ) is brittle and tied to specific legacy network environments.
- Tightly Coupled Infrastructure:** Database initialization logic ( `init_db.js` ) runs inside the application process on startup, conflating deployment concerns with runtime logic.

### Non-Trivial Decisions Found

---

- Custom Config Loader:** Instead of using standard libraries like `dotenv` or `node-config`, the team wrote a custom `switch` statement in `config.js` to handle `LOCAL`, `DOCKER`, and `DEVEL` environments.
- "Dual-Mode" Buy Route:** The `/products/buy` route in `routes/products.js` accepts `router.all`, handling logic for both `GET` (query params) and `POST` (body). This suggests a

decision to support purchase via direct URL links (CSRF risk) or a lack of understanding of HTTP verbs.

- **In-Memory Session Store:** The `express-session` is configured without a persistent store (e.g., Redis), meaning all user sessions are lost if the application restarts.

## Modernization Documentation

---

For a deep dive into the modernization plan, please refer to the following documents:

- [ONBOARDING\\_LOG.md](#) - Analysis of the developer experience and setup friction.
- [CONTEXT\\_MAP.md](#) - Domain-Driven Design analysis of the business boundaries.
- [USER\\_STORIES.md](#) - Reverse-engineered backlog of current functionality.